

体系结构设计实践与验证

软件工程与计算 II · 体系结构模块 · 第 3 讲

南京大学 · 刘钦

今日学习路径

① 逻辑视角 → ② 4+1视图 → ③ 接口设计 → ④ 构建落地 → ⑤ 集成验证 → ⑥ AI辅助

包设计原则

多视角建模

DIP与DI

代码映射

集成策略

LLM辅助架构



今日核心问题

1. 如何将**用例分析**结果系统地映射为**包结构**？
2. 如何通过**接口设计**与**依赖倒置**实现层间解耦？
3. 如何选择**集成策略**并验证体系结构的正确性？

预习测试（5 分钟）

请在纸上完成以下两题：

1. 简述体系结构设计的**过程步骤**（至少写出 4 步）
2. 画出学生选课业务逻辑 `chooseCoursebl` 的**关键类图**，标注类之间的依赖关系

提示：回顾第 2 讲中分层体系结构的设计方法

PART 1 · 逻辑视角设计

从用例到包结构的系统映射

逻辑视角与包的概念

逻辑视角

- 关注系统的**功能组织**
- 描述系统的**静态结构**
- 核心单位：**包 (Package)**
- 目标：高内聚、低耦合

包 (Package)

- UML 中的**分组机制**
- 将相关的类/接口组织在一起
- 对应 Java 中的 `package`
- 包之间存在**依赖关系**

包是体系结构逻辑视角的基本构造单元，好的包划分直接决定系统的可维护性。

从用例到包的 4 步映射方法

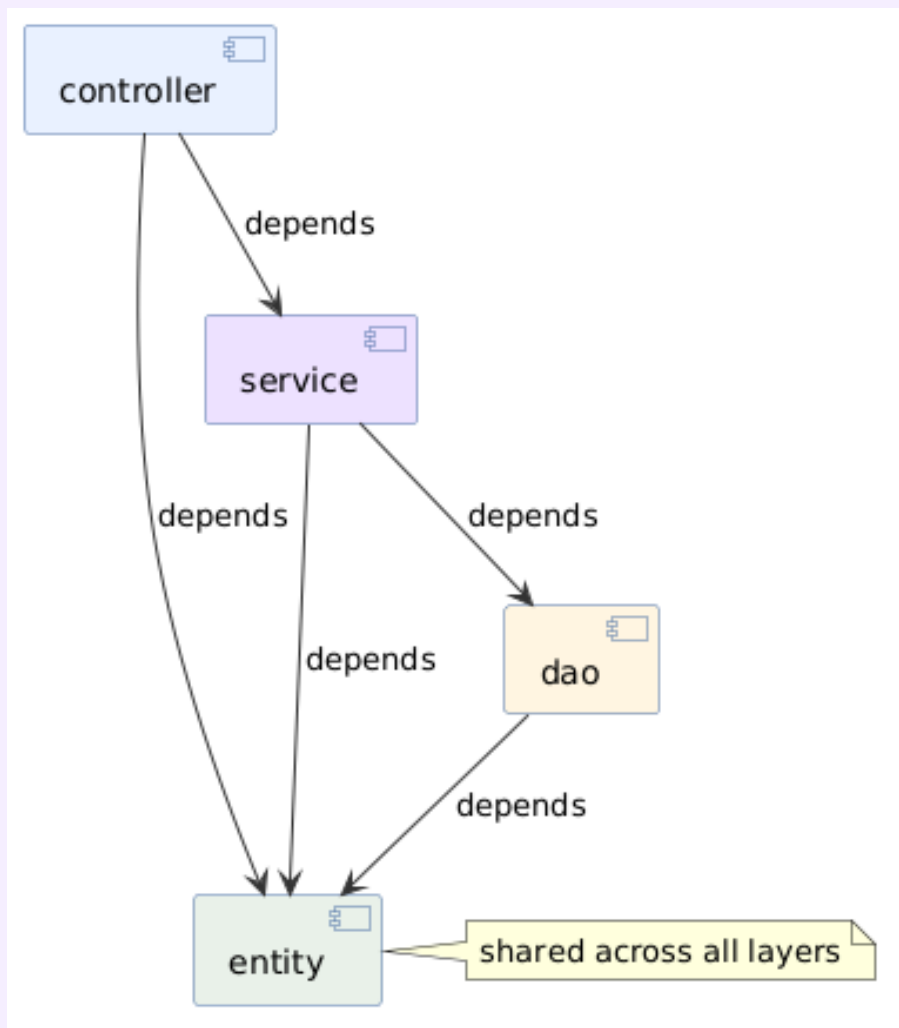
步骤	活动	产出
Step 1	确定顶层包结构	controller / service / dao / entity 四层
Step 2	分析用例，确定业务模块	book / cart / order / customer / payment / inventory / review
Step 3	将概念类分配到 entity 包	Book, Order, Customer, CartItem 等
Step 4	为每层建立与业务模块对应的类	BookController, BookService, BookDao 等

原则：先纵向分层，再横向分模块，确保每个包职责单一。

案例：在线书店完整包结构

```
com.bookstore
├── controller
│   ├── BookController
│   ├── CartController
│   ├── OrderController
│   └── CustomerController
├── service
│   ├── BookService      (接口)
│   ├── CartService      (接口)
│   ├── OrderService     (接口)
│   ├── InventoryService (接口)
│   └── impl/
│       ├── BookServiceImpl
│       ├── CartServiceImpl
│       └── OrderServiceImpl
├── dao
│   ├── BookDao, CartDao, OrderDao, CustomerDao (接口)
│   └── impl/ ...
└── entity
    ├── Book, Order, OrderItem, CartItem, Customer, Review
```

包的依赖关系



关键约束：依赖方向**单向向下**，禁止反向依赖或跨层调用。

包设计原则 — 内聚性三原则

原则	全称	含义
REP	重用-发布等价原则	重用的粒度 = 发布的粒度
CRP	共同重用原则	一起被重用的类放同一个包
CCP	共同封闭原则	因同一原因变更的类放同一个包

书店案例

- REP: `entity` 包整体发布, `Book` 和 `Order` 一起被所有层引用
- CRP: `BookDao` 和 `BookService` 总是一起使用 → 放在同一业务模块下
- CCP: 支付需求变更只影响 `payment` 模块, 不波及 `book` 模块

包设计原则 — 耦合性三原则

原则	全称	含义
ADP	无环依赖原则	包之间的依赖关系不能形成环
SDP	稳定依赖原则	依赖方向朝着更稳定的包
SAP	稳定抽象原则	越稳定的包应越抽象

书店案例

- **ADP**: controller → service → dao, 无环
- **SDP**: `entity` 最稳定 (很少变化), 所有层都依赖它
- **SAP**: `dao` 包对外暴露的是**接口** (抽象), 实现类放在 `dao.impl` 中

REP 重用-发布等价原则 [Robert C. Martin]

The granule of reuse is the granule of release.

重用的单元 = 发布的单元。

含义

- 如果你要复用一个包中的某个类，你就得依赖**整个包**
- 因此，包中的类必须有**理由被一起重用**

案例：在线书店

包	是否满足 REP	理由
<code>entity</code> (Book, Order, Customer...)	满足	所有层都引用 <code>entity</code> ，整体发布
假如把 <code>Book</code> 和 <code>PaymentGateway</code> 放同一包	违反	用图书搜索功能不需要支付网关

CRP 共同重用原则 [Robert C. Martin]

Classes that are used together belong together.
一起被使用的类应放在一起。

含义

- CRP 告诉我们哪些类**不该**放在一起：如果用了包中 A 类但不需要 B 类，说明 A 和 B 不该在同一个包

案例：连锁超市

功能	对应逻辑包	CRP 分析
销售	SalesUI, Sales, SalesData	三个总是一起使用 ✓
库存	CommodityUI, Commodity, CommodityData	三个总是一起使用 ✓
销售+库存放一个包?	违反 CRP	修改库存时被迫重新测试销售

CCP 共同封闭原则 [Robert C. Martin]

Classes that change together belong together.

因同一原因变化的类应放在一起（包级别的 SRP）。

含义

- 当需求变更时，变更应该集中在尽可能少的包中
- 违反 CCP = 一次需求变更要改多个包 = 高维护成本

案例：在线书店

需求变更："支付方式新增微信支付"

如果按 CCP 划分	变更范围
<code>service.payment</code> + <code>dao.payment</code>	只改 2 个包 ✓
如果支付和订单混在一起	改动波及订单逻辑 ✗

CCP 与 CRP 的权衡 [Robert C. Martin]

CCP 和 CRP 是**互斥的** —— 不可能同时完美满足两者。

维度	CCP（共同封闭）	CRP（共同重用）
服务对象	维护者（Maintainers）	重用者（Reusers）
包的大小	倾向让包 更大 （变更集中）	倾向让包 更小 （减少不必要依赖）
优先场景	项目 早期 ：需求频繁变更	项目 成熟期 ：对外发布/复用

项目生命周期中的演进策略

- **早期**：以 CCP 为主导 → 大包、便于开发和维护
- **后期**：架构稳定后，重构包结构 → 以 CRP 为主导，方便外部复用

没有"正确答案"，只有**当前阶段最合适的权衡**

SDP 稳定依赖原则 [Robert C. Martin]

Depend in the direction of stability.

依赖方向应朝着更稳定的方向。

怎么判断稳定性？

- **稳定** = 很多包依赖它（被需要）→ 改动代价大 → 不容易变
- **不稳定** = 它依赖很多包 → 容易变

案例：不稳定性计算

包	Ca(传入)	Ce(传出)	$I = Ce / (Ca + Ce)$	稳定性
entity	4	0	0.00	最稳定 — 正确！
controller	0	3	1.00	最不稳定 — 正确！
service	1	2	0.67	中等

controller（不稳定）→ service（中等）→ dao → entity（稳定）✓ 符合 SDP

SAP 稳定抽象原则 [Robert C. Martin]

A package should be as abstract as it is stable.

越稳定的包应越抽象；越不稳定的包应越具体。

SDP + SAP = 依赖倒置原则（DIP）在包级别的体现

包	稳定度	应有的抽象度	实际做法
entity	最稳定 (I=0)	高抽象	POJO, 无实现细节 ✓
dao (接口包)	较稳定	高抽象	只有 interface ✓
controller	最不稳定 (I=1)	低抽象	全是具体实现 ✓
service (接口包)	中等	中等	interface + 部分实现 ✓

痛苦区：稳定但具体（改不动但想改） → 要重构为接口

无用区：不稳定但抽象（没人用的接口） → 可以删除

案例：连锁超市 — 从用例到包的完整过程

步骤 1：识别基本功能（从用例图）

功能模块	包含的用例
销售	销售处理、退货
库存	商品入库、出库、产品调整、库存分析
会员	发展会员、礼品赠送
销售策略	特价策略制定、赠送策略制定
用户	用户管理

步骤 2：建立初始包结构（粗粒度三层）

选择分层风格 → 每个功能在 3 层各建一个包 → 共 $5 \times 3 = 15$ 个包

初始概要功能设计

功能	展示层包	业务逻辑层包	数据层包
销售	SalesUI	Sales	SalesData
库存	CommodityUI	Commodity	CommodityData
会员	MemberUI	Member	MemberData
销售策略	PromotionUI	Promotion	PromotionData
用户	UserUI	User	UserData

关键问题

销售是否只用到 Sales 的数据？

改进：发现横向依赖

分析销售处理用例的数据需求

销售处理不仅需要 Sales 数据，还需要：

- **Commodity** + CommodityData（查询商品信息、价格）
- **Member** + MemberData（查询会员折扣）
- **Promotion** + PromotionData（应用促销策略）
- **User** + UserData（验证收银员身份）

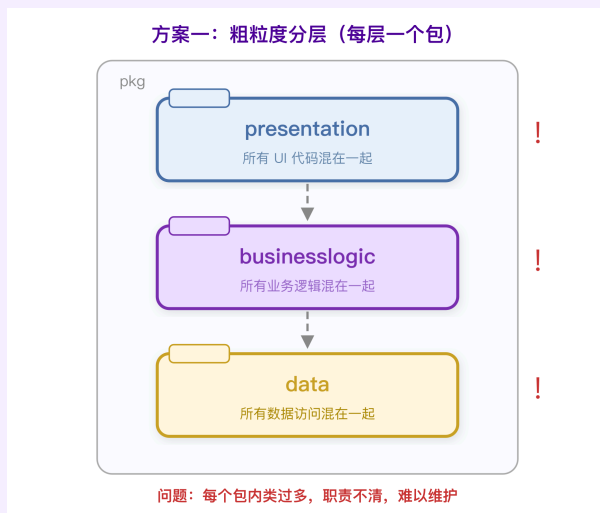
改进后的概要功能设计

功能	逻辑包（黑色=本模块，红色=跨模块依赖）
销售	Sales, SalesData; + Commodity, Member, Promotion, User
库存	Commodity, CommodityData; + Sales, SalesData
会员	Member, MemberData; + Sales, Commodity

横向调用是不可避免的，但需要通过接口来管理

设计方案对比

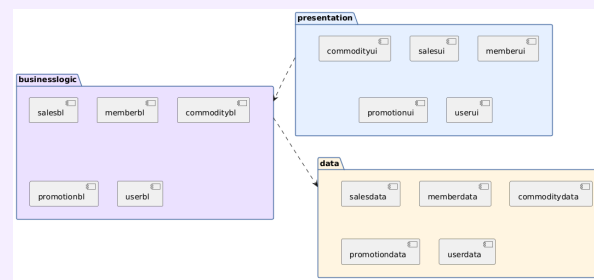
方案一：粗粒度（每层一个包）



问题：

- 展示层一个大包 → 类太多
- 数据层只有 Database 包 → 无法独立替换
- presentation 直接跨层调用 data

方案二：细粒度（按功能模块拆包）



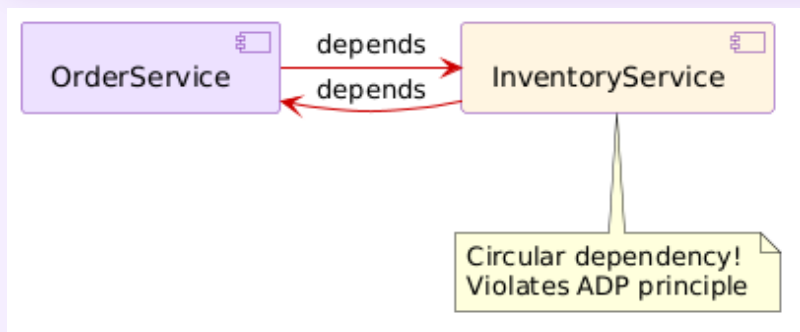
优点：

- 每个包职责明确（CCP）
- 业务逻辑层内部横向调用清晰
- 数据层按模块拆分，可独立替换
- presentation → businesslogic → data 单向

案例：ADP 违规 — 循环依赖

问题场景

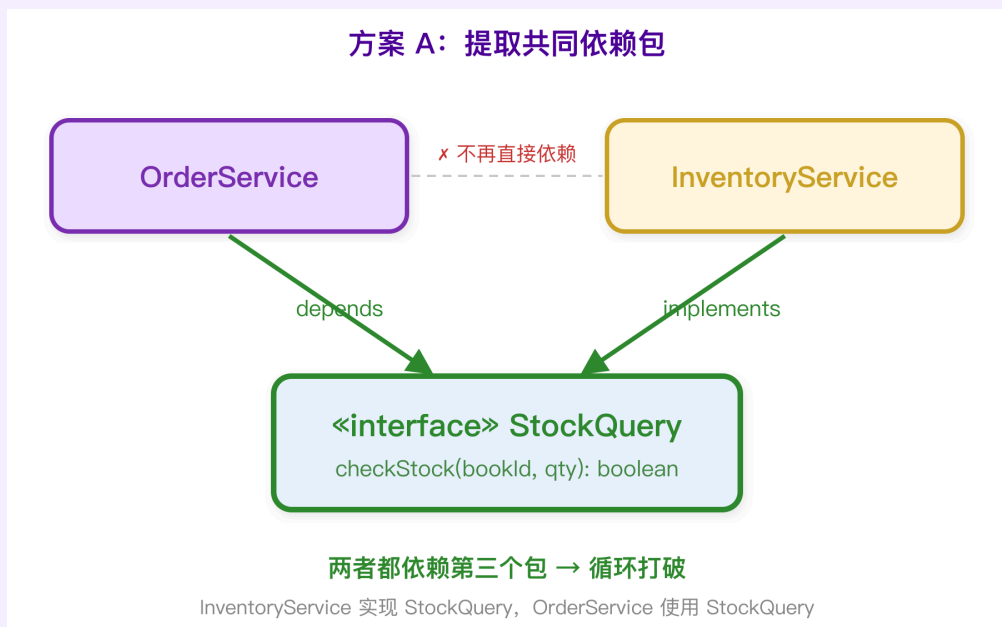
OrderService 调用 InventoryService.checkStock()
InventoryService 调用 OrderService.getOrderQty()



危害

- 两个模块无法独立编译/测试
- 修改一方会连锁影响另一方
- 违反 ADP 原则

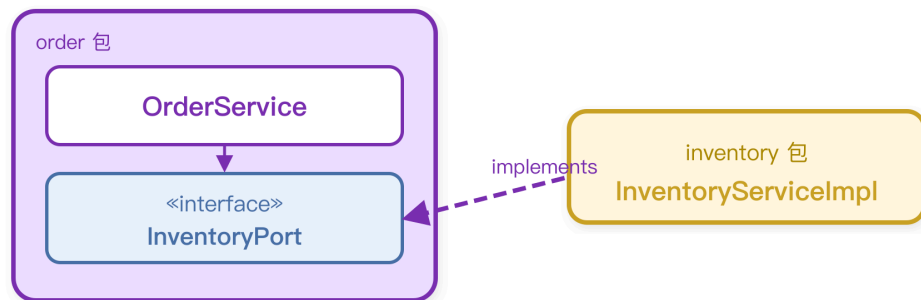
ADP 修复方案 A：提取共同依赖包



- 新建独立的**接口包**（如 `stock-api`），包含 `StockQuery` 接口
- `OrderService` 和 `InventoryService` 都依赖这个第三方包
- 原来的 $A \rightleftarrows B$ 循环变成 $A \rightarrow C \leftarrow B$ ，循环打破

ADP 修复方案 B：接口定义在调用方包内（反转依赖方向）

方案 B：接口定义在调用方包内（反转依赖方向）



依赖方向反转：inventory → order（而非 order → inventory）
接口定义在调用方(order)，实现方(inventory)反过来依赖调用方的接口

原来：order → inventory（循环）→ 现在：inventory → order（单向）

- 在 **order 包内**定义 **InventoryPort** 接口
- **InventoryServiceImpl**（inventory 包）**反过来依赖 order 包**的接口
- 依赖方向从 order→inventory 反转为 inventory→order，循环打破

方案 A = 提取第三方；方案 B = 反转方向。核心都是面向接口编程。

逻辑视角设计小结

要点	说明
映射方法	4 步法：分层 → 分模块 → 分配实体 → 建立各层类
依赖方向	单向向下：controller → service → dao → entity
内聚性原则	REP / CRP / CCP — 决定哪些类放在一起
耦合性原则	ADP / SDP / SAP — 决定包之间如何依赖
核心武器	接口（抽象）是打破耦合与循环依赖的关键

PART 2 · 4+1 视图

多视角理解体系结构

View vs Viewpoint [SWEBOK 2.1]

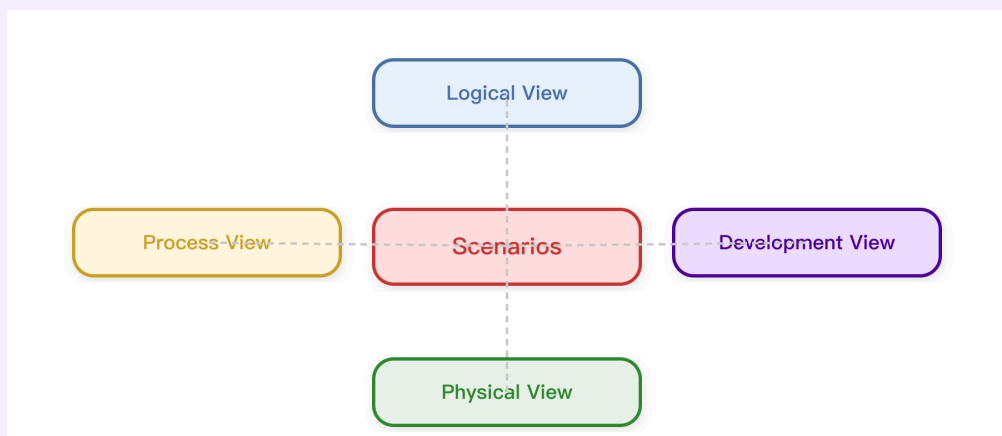
概念	定义	类比
Viewpoint (视点)	一套约定和规范, 指导如何创建和使用某类视图	建筑制图标准 (平面图怎么画)
View (视图)	从某个视点出发, 对系统架构的一次具体表达	一张具体的建筑平面图

常见视点 [Rozanski & Woods]

视点	关注点	受众
Module	模块组织、依赖	开发者
Component & Connector	运行时交互	架构师
Allocation	部署、资源分配	运维

一个视点可以产出多个视图; ADL (如 UML) 是表达视图的工具

Kruchten 4+1 视图模型

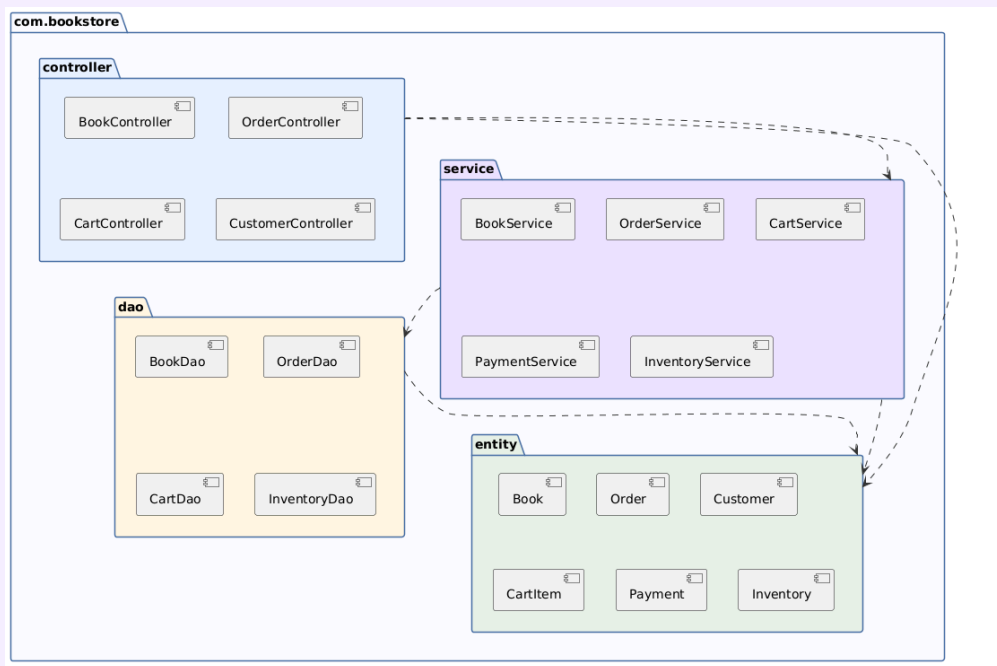


每个视图面向不同的**利益相关者**，解决不同的关注点。

五个视图对照表

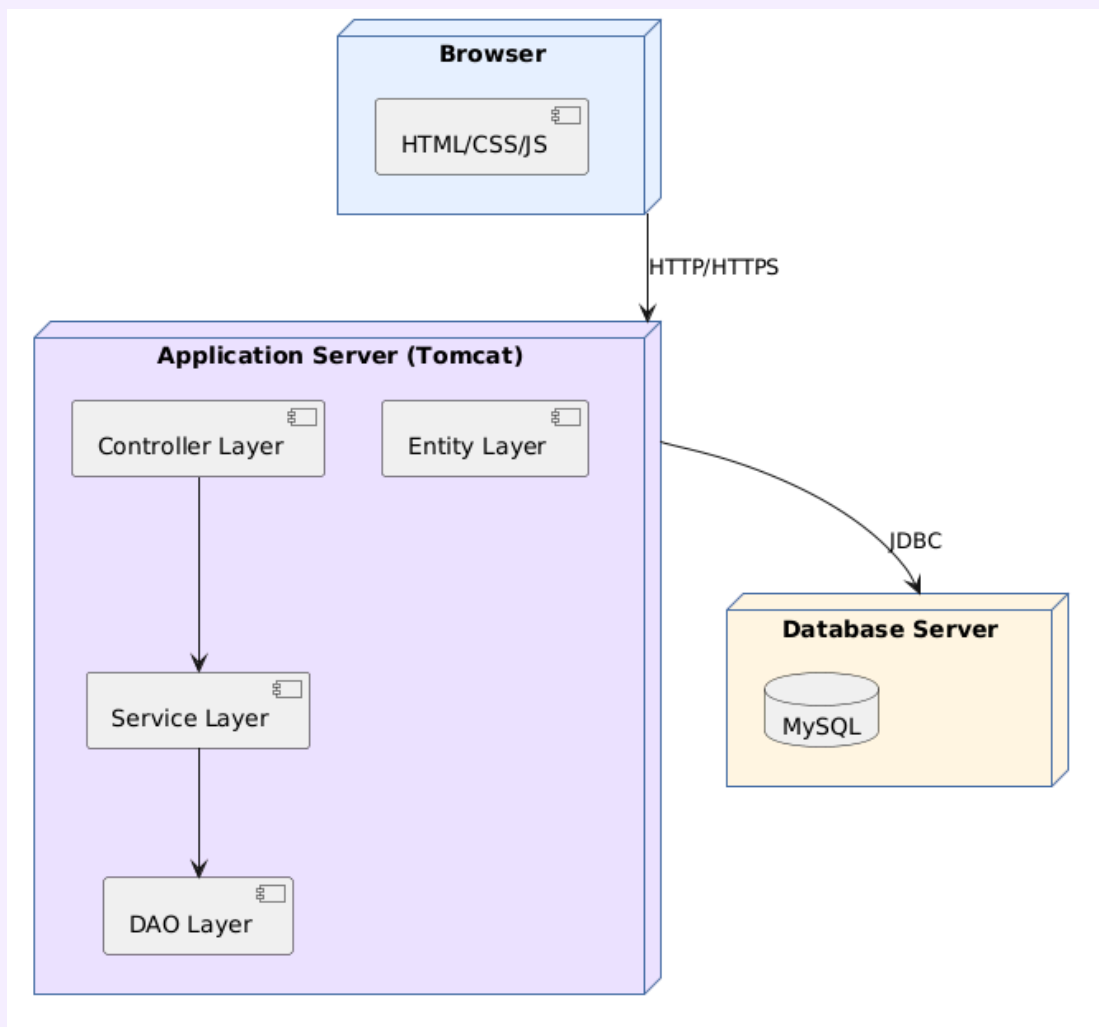
视图	关注点	对应 UML 图	面向角色
逻辑视图	功能需求、类与包的组织	包图、类图	架构师、开发者
开发视图	代码组织、模块划分	组件图	开发者
进程视图	并发、同步、性能	活动图、序列图	系统集成者
物理视图	部署拓扑、硬件映射	部署图	运维工程师
场景视图	用例驱动、验证其他视图	用例图	所有利益相关者

案例：在线书店 — 逻辑视图（包图）



每个顶层包内部按**业务模块**再次划分（book / order / cart / customer）。

案例：在线书店 — 物理视图（部署图）



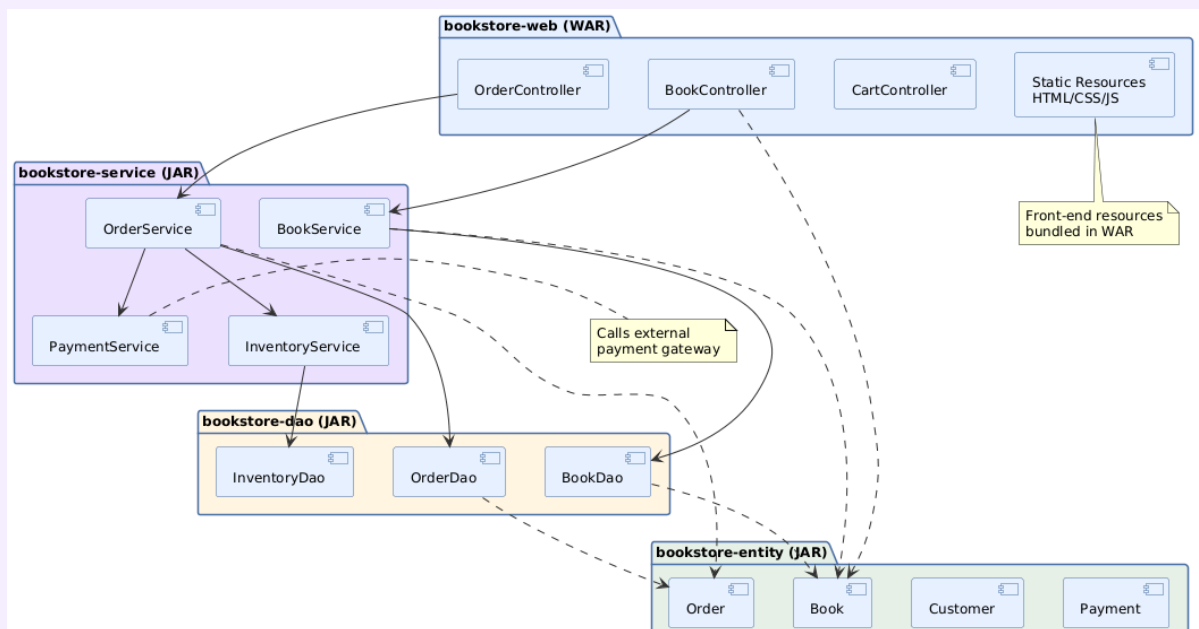
四层代码全部部署在同一个 Tomcat 进程中（单体架构）。

案例：在线书店 — 开发视图（组件图）

代码组织 → 构建单元 → 交付物

模块	Maven 模块 / 包	构建产出	负责团队
bookstore-web	controller + 静态资源	WAR 入口	前端 + A
bookstore-service	service 接口 + 实现	JAR	后端 B/C
bookstore-dao	dao 接口 + MyBatis mapper	JAR	DBA E
bookstore-entity	实体类 + 枚举	JAR（公共依赖）	全员共享

UML 组件图



案例：在线书店 — 进程视图

关注并发、线程、同步

单体部署时的线程模型

线程/进程	职责
Tomcat Worker Thread Pool	处理 HTTP 请求 (200 线程)
HikariCP Connection Pool	数据库连接池 (max=20)
Scheduler Thread	定时任务 (订单超时取消)
Async Thread Pool	异步邮件/短信通知

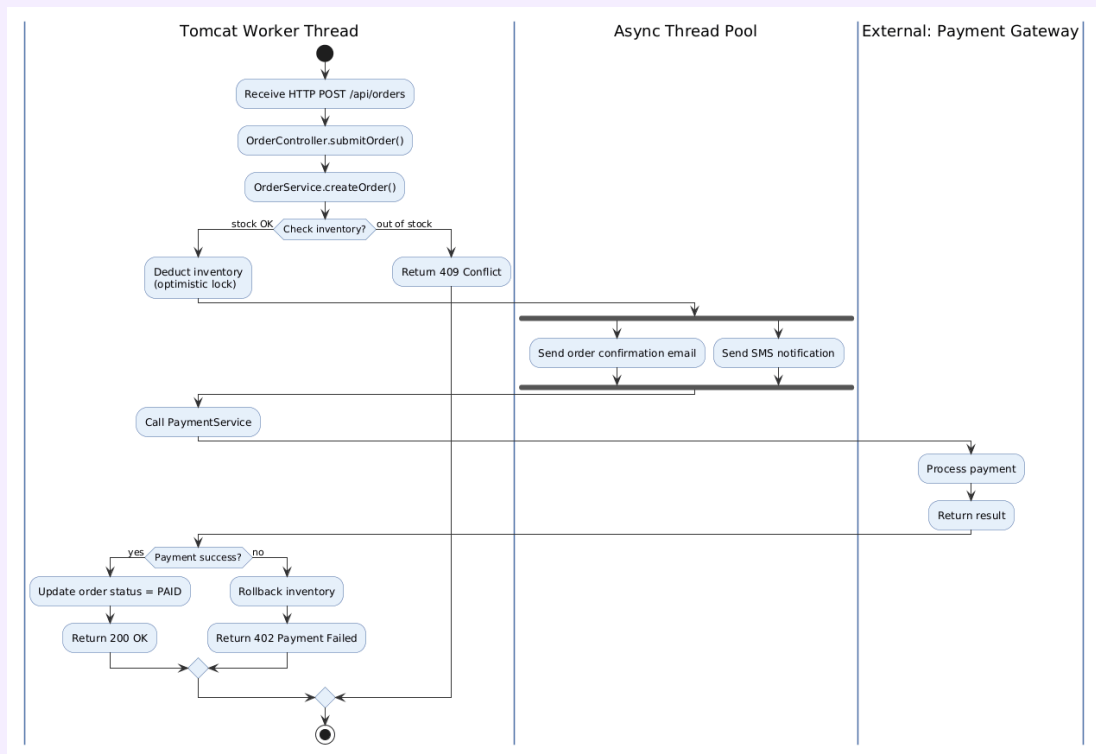
并发关注点

- 库存扣减：乐观锁 `UPDATE inventory SET qty=qty-1 WHERE qty>0`
- 订单超时：Scheduler 每分钟扫描超时订单
- 支付回调：幂等处理，防止重复扣款
- 缓存一致：Redis 缓存与 DB 的最终一致性

进程视图回答："系统在运行时如何分配线程和资源？"

案例：在线书店 — 进程视图（活动图）

"购买图书"用例的线程/进程活动图



活动图展示了请求在**多线程/多进程**间的流转：Worker Thread → Async Pool → 外部支付网关

案例：在线书店 — 场景视图（用例驱动）

场景视图 = 用例 × 架构各层的穿透

用例	展示层	业务层	数据层	并发/部署关注点
购买图书	OrderController	OrderService → PaymentService	OrderDao, InventoryDao	库存乐观锁，支付幂等
搜索图书	BookController	BookService	BookDao (+ Redis 缓存)	缓存命中率，全文索引
管理购物车	CartController	CartService	CartDao	Session/Token 一致性
查看报表	ReportController	ReportService	OrderDao (聚合查询)	慢查询优化，异步导出

场景视图是验证视图——通过走查关键用例，检验逻辑/开发/进程/物理四个视图是否协调一致

4+1 视图总结

视图	核心问题	在线书店案例要点
逻辑	功能怎么划分?	controller / service / dao / entity 四层包
开发	代码怎么组织和构建?	4 个 Maven 模块, entity 是公共依赖
进程	运行时线程/并发如何?	200 worker + 20 DB连接 + 定时器 + 异步池
物理	部署在哪里?	Browser → Tomcat → MySQL (单体)
场景	用例能否穿透所有层?	购买图书穿透 4 层 + 库存锁 + 支付幂等

场景视图是粘合剂——如果一个用例无法顺畅穿透四个视图, 说明架构有问题

从逻辑层到物理节点的映射

逻辑层	职责	部署位置	技术选型
controller	接收请求、参数校验	Tomcat	Spring MVC
service	业务逻辑、事务管理	Tomcat	Spring Service
dao	数据持久化	Tomcat	MyBatis / JPA
entity	数据实体定义	Tomcat	POJO
前端	页面渲染、用户交互	浏览器	HTML/CSS/JS
数据库	数据存储	MySQL 服务器	MySQL 5.7+

逻辑分层 $\neq$ 物理分离。单体架构中逻辑层共享同一进程。

PART 3 · 接口设计与依赖倒置

用抽象构建体系结构的"楼板"

接口在体系结构中的作用

"楼板"比喻



接口是层与层之间的**契约**，如同楼板将楼层隔离。

接口的三大好处

1. **解耦**：上层不依赖下层实现
2. **可替换**：实现类可以随时替换
3. **可测试**：用 Mock 替代真实实现

先定义接口，再编写实现 — 这就是**面向接口编程**。

PO 与 VO 概念

概念	全称	用途	所在层
PO	Persistent Object	与数据库表一一对应	entity / dao
VO	Value Object	面向展示层的数据传输对象	controller / service

为什么需要区分？

- PO 包含数据库细节（主键、外键、时间戳）
- VO 只包含前端需要的字段
- **隔离变化**：数据库字段变更不影响前端接口

```
// PO: 对应 book 表
public class Book { Long id; String isbn; String title; BigDecimal price; ... }

// VO: 前端展示用
public class BookVO { String isbn; String title; String priceText; double rating; }
```

依赖倒置原则（DIP）详解

传统依赖

高层模块 —依赖—▶ 低层模块
(service) (dao 实现)

高层直接依赖低层**具体实现**，耦合度高。

DIP 两条规则：

1. 高层模块不应依赖低层模块，两者都应依赖抽象
2. 抽象不应依赖细节，细节应依赖抽象

依赖倒置后

高层模块 —依赖—▶ 抽象接口
低层模块 —实现—┘
▲

高层和低层都依赖**抽象**，依赖方向"倒置"。

案例：BookDao 接口设计

```
public interface BookDao {  
  
    Book findById(Long id);  
  
    Book findByIsbn(String isbn);  
  
    List<Book> search(String keyword, int page, int size);  
  
    void save(Book book);  
  
    void update(Book book);  
  
    void deleteById(Long id);  
  
    List<Book> findByCategory(String category);  
}
```

接口只定义做什么（What），不涉及怎么做（How）。实现可以是 MyBatis、JPA 或内存 Map。

案例：BookService 接口设计

```
public interface BookService {  
  
    BookVO findById(Long id);  
  
    List<BookVO> search(String keyword, int page, int size);  
  
    boolean checkStock(Long bookId, int quantity);  
  
    void addBook(BookVO bookVO);  
  
    void updateBook(BookVO bookVO);  
}
```

设计要点

- 入参和返回值使用 VO（面向上层），内部再转换为 PO
- `checkStock` 方法体现**业务逻辑**：不只是查数据，还要判断库存是否充足
- 方法粒度对应**用例步骤**，而非数据库 CRUD

案例：BookServiceImpl — 构造器注入

```
@Service
public class BookServiceImpl implements BookService {

    private final BookDao bookDao;

    // 构造器注入 — Spring 推荐方式
    @Autowired
    public BookServiceImpl(BookDao bookDao) {
        this.bookDao = bookDao;
    }

    @Override
    public BookVO findById(Long id) {
        Book book = bookDao.findById(id);
        return convertToVO(book);
    }

    @Override
    public boolean checkStock(Long bookId, int quantity) {
        Book book = bookDao.findById(bookId);
        return book != null && book.getStock() >= quantity;
    }
}
```

无状态 Service 设计

正确做法（无状态）

```
@Service
public class BookServiceImpl
    implements BookService {

    private final BookDao bookDao;

    @Autowired
    public BookServiceImpl(BookDao dao) {
        this.bookDao = dao;
    }

    public BookVO findById(Long id) {
        Book b = bookDao.findById(id);
        return convertToV0(b);
    }
}
```

错误做法（有状态）

```
@Service
public class BookServiceImpl
    implements BookService {

    private Book currentBook; // 危险!

    public void loadBook(Long id) {
        this.currentBook =
            bookDao.findById(id);
    }

    public BookVO getCurrent() {
        return convertToV0(currentBook);
    }
}
```

有状态的 Service 在多线程环境下会导致数据混乱！ Spring 默认单例模式，多个请求共享同一个实例。

接口设计小结

设计要点	说明
接口 = 契约	定义层间交互的 What，隐藏 How
PO vs VO	隔离持久化细节与展示需求
DIP	高层和低层都依赖抽象，不直接耦合
构造器注入	Spring 推荐的 DI 方式，保证不可变性
无状态设计	Service 不持有请求级别的状态，确保线程安全

接口设计质量直接决定体系结构的可演化性和可测试性。

PART 4 · 体系结构构建

从设计到代码的落地

从设计到代码的落地

核心映射关系

设计概念	代码实现
包 (Package)	Java package / 目录
接口	Java interface
依赖关系	import 语句 / 构造器参数
层 (Layer)	顶层 package 划分
业务模块	子 package 划分

落地原则

1. **目录结构 = 包结构**：代码组织严格反映架构设计
2. **依赖检查**：import 只能指向允许的层
3. **接口先行**：先建接口文件，再写实现类

案例：Spring Boot 项目结构

```
bookstore/
├── src/main/java/com/bookstore/
│   ├── BookstoreApplication.java      ← 启动类
│   ├── controller/
│   │   ├── BookController.java
│   │   └── OrderController.java
│   ├── service/
│   │   ├── BookService.java          ← 接口
│   │   ├── OrderService.java        ← 接口
│   │   └── impl/
│   │       ├── BookServiceImpl.java
│   │       └── OrderServiceImpl.java
│   ├── dao/
│   │   ├── BookDao.java              ← 接口
│   │   └── OrderDao.java             ← 接口
│   └── entity/
│       ├── Book.java
│       └── Order.java
├── src/main/resources/
│   └── application.properties
└── pom.xml
```


案例：Spring Boot 配置

```
# application.properties

# 服务端口
server.port=8080

# 数据库连接
spring.datasource.url=jdbc:mysql://localhost:3306/bookstore
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# MyBatis 配置
mybatis.mapper-locations=classpath:mapper/*.xml
mybatis.type-aliases-package=com.bookstore.entity

# 日志级别
logging.level.com.bookstore=DEBUG
```

配置文件体现了物理视图的关键信息：端口、数据库地址、技术选型。

PART 5 · 集成策略与验证

确保体系结构设计的正确落地

集成策略对比

策略	方式	优点	缺点
大爆炸	所有模块一次集成	简单直接	难以定位缺陷
自顶向下	从 controller 层开始	早期验证主流程	需要大量 Stub
自底向上	从 dao 层开始	底层测试充分	需要大量 Driver
三明治	顶层+底层同时，中间最后	平衡两者优点	中间层测试较晚
持续集成	每次提交触发集成	快速反馈	需要自动化基础设施

Stub 和 Driver 的概念

Stub（桩）

- 替代**被调用**的下层模块
- 模拟下层返回固定数据
- 用于**自顶向下**集成

```
// BookDao 的 Stub
public class BookDaoStub
    implements BookDao {
    public Book findById(Long id) {
        return new Book(1L, "978-7-XXX",
            "设计模式", new BigDecimal("59"));
    }
}
```

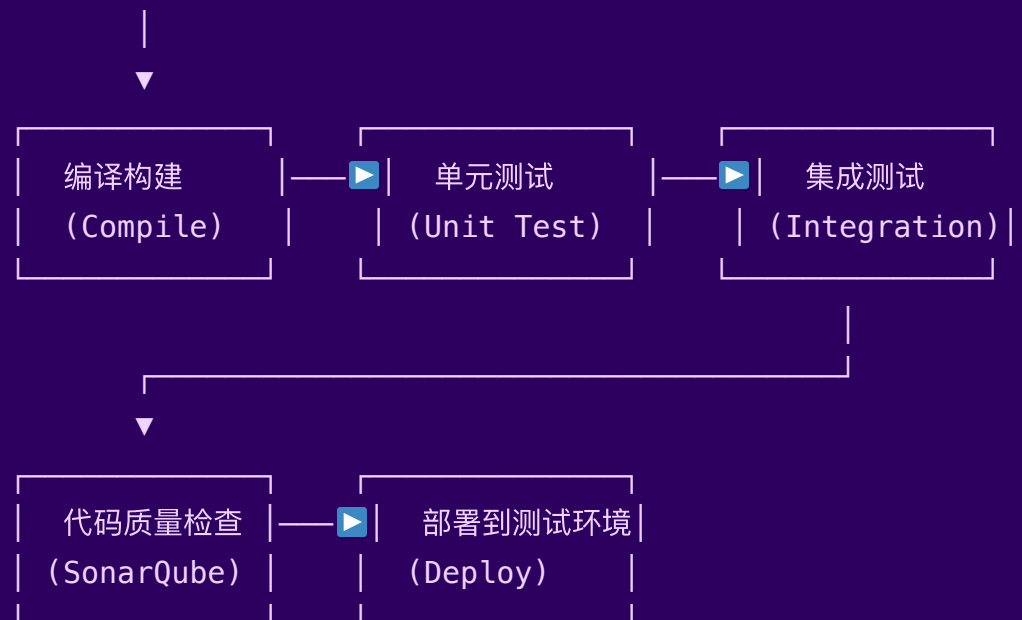
Driver（驱动）

- 替代**调用者**的上层模块
- 模拟上层发起请求
- 用于**自底向上**集成

```
// 测试 BookDao 的 Driver
public class BookDaoDriver {
    public static void main(String[] args) {
        BookDao dao = new BookDaoImpl();
        Book book = dao.findById(1L);
        assert book != null;
        System.out.println("测试通过");
    }
}
```

持续集成 (CI) 流程

开发者提交代码



CI 的核心价值

- **快速反馈**: 提交后几分钟内知道是否破坏了集成
- **降低风险**: 频繁小集成 vs 一次大集成
- **工具链**: Git + Jenkins/GitHub Actions + Maven/Gradle

体系结构评审方法

ATAM

Architecture Tradeoff Analysis Method

- 分析**质量属性之间的权衡**
- 输入：架构文档 + 质量属性场景
- 输出：风险点、权衡点、敏感点
- 适用于**大型系统**早期评审

SAAM

Software Architecture Analysis Method

- 基于**场景**的分析方法
- 将场景分为**直接场景**和**间接场景**
- 评估架构对场景的支持程度
- 较 ATAM 更简单，适合**中小型**项目

两种方法都强调：在编码之前发现架构问题，修复成本远低于编码之后。

集成测试验证体系结构正确性

```
@SpringBootTest
@AutoConfigureMockMvc
public class BookIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    public void testSearchBooks() throws Exception {
        // 验证 controller → service → dao 完整链路
        mockMvc.perform(get("/api/books/search")
            .param("keyword", "设计模式"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.length()").value(greaterThan(0)));
    }
}
```

集成测试验证的架构属性

- **层间连通性**：请求能否正确穿透所有层
- **依赖注入**：Spring 容器是否正确装配所有 Bean
- **事务管理**：跨层操作的事务是否正确传播

集成验证小结

阶段	方法	目的
设计阶段	ATAM / SAAM 评审	发现架构风险与权衡
编码阶段	持续集成 CI	快速反馈集成问题
测试阶段	Stub / Driver	隔离测试各层
测试阶段	集成测试	验证端到端链路

验证贯穿全生命周期，越早发现问题，修复成本越低。

"好架构"的判断标准 [SWEBOK 4.1]

古罗马建筑师 Vitruvius 认为好建筑应具备：**firmitas**（坚固）、**utilitas**（实用）、**venustas**（美观）

评估好架构的四个问题

#	问题	在线书店评估
1	是否在其生命周期内足够 健壮 ，能应对演化？	分层 + 接口隔离 → 可替换数据层
2	是否 适用于 其预期用途？	三层架构满足 Web 电商场景
3	用此架构构建系统是否 经济可行 ？	5 人团队用 Spring Boot，成本可控
4	对必须构建/维护它的人来说，是否 清晰可理解 ？	标准分层 + 标准命名，新人可快速上手

架构度量指标 [SWEBOK 4.4]

包级度量

指标	公式	含义
Ca (传入依赖)	依赖本包的外部包数	越高越稳定
Ce (传出依赖)	本包依赖的外部包数	越高越不稳定
I (不稳定性)	$Ce / (Ca + Ce)$	0=完全稳定, 1=完全不稳定
A (抽象度)	抽象类数 / 总类数	接口包→高, 实现包→低

案例：在线书店包度量

包	Ca	Ce	I	A	评估
entity	4	0	0.00	0.0	最稳定, 符合 SDP
dao (接口)	2	1	0.33	1.0	稳定+抽象, 符合 SAP
service (接口)	1	2	0.67	1.0	接口抽象, 依赖适中
controller	0	3	1.00	0.0	最不稳定, 只有实现

DevOps 度量：变更前置时间、部署频率、MTTR、变更失败率——也是架构健康度的指标

ADL —— 架构描述语言 [SWEBOK 2.3]

- **ADL (Architecture Description Language) **是专门用于表达软件体系结构的领域特定语言
- 起源于模块互连语言 (MIL), 如 C2、Rapide、ACME、Wright
- 工业界实际: **UML 已成为事实标准 ADL**, 因其广泛使用

本课程使用的 ADL 工具

工具	用途	示例
PlantUML	包图、组件图、部署图	@startuml ... package ... @enduml
Mermaid	流程图、时序图	graph TD; A-->B
Spring Boot 目录结构	架构的代码级表达	com.bookstore.controller/service/dao

Architecture Framework (如 AUTOSAR、UAF) 是更系统化的架构描述框架

PART 6 · AI 辅助体系结构

LLM 赋能架构设计

LLM 集成系统的参考架构 [arxiv 2501.12904, 2025]

功能参考架构 (Functional Reference Architecture)

层	职责	典型组件
Interaction Layer	用户交互、Prompt 管理	Chat UI、Prompt Template
Orchestration Layer	任务编排、Agent 协调	LangChain、CrewAI
Model Layer	LLM 推理、模型管理	GPT-4、Claude、本地模型
Data Layer	向量存储、知识检索	Pinecone、Chroma、RAG
Infrastructure Layer	部署、监控、安全	Docker、K8s、Guardrails

验证案例

- MaxKB：知识库问答系统
- Continue：IDE 代码助手
- InternVL：多模态视觉语言系统

这个参考架构帮助团队在构建 AI 应用时做出一致的架构决策

课堂讨论：AI 能否取代架构师？

正方：AI 可以做架构

- 秒级生成完整项目骨架
- 一致性高，不会忘记规范
- 能分析大规模依赖图
- 知识面广，熟悉多种风格

反方：AI 无法取代架构师

- 缺乏**业务上下文**理解
- 无法权衡**组织/政治/成本**因素
- 架构决策需要**长期视角**
- 不理解**利益相关人**的真实关注点

结论：AI 是架构师的**放大器**，不是替代者。

"Humans own architecture, AI validates it."

LLM 生成 Spring Boot 项目骨架

Prompt 示例

请为"在线书店系统"生成 Spring Boot 项目骨架:

- 分层架构: `controller / service / dao / entity`
- 业务模块: `book, order, cart, customer`
- 每层为每个模块生成接口和实现类
- 使用构造器注入
- 包名: `com.bookstore`

请输出完整的目录结构和关键接口代码。

LLM 的价值与局限

价值	局限
快速生成模板代码	可能不符合项目特定约定
减少重复劳动	缺乏业务深度理解
提供参考架构	需要人工审查和调整

AI 辅助包依赖分析

Prompt 示例

分析以下 Java 包结构的依赖关系，检查是否存在：

1. 循环依赖
2. 违反分层约束的依赖（如 dao 层依赖 controller）
3. 不合理的跨模块依赖

包结构：

```
com.bookstore.controller.book → com.bookstore.service.book  
com.bookstore.service.book → com.bookstore.dao.book  
com.bookstore.service.order → com.bookstore.service.book  
com.bookstore.dao.order → com.bookstore.service.inventory ← 可疑！
```

AI 检测结果

违规：`dao.order` 依赖 `service.inventory` — dao 层不应依赖 service 层，违反分层约束。应将库存查询逻辑提升到 service 层完成。

课堂实操 (20 分钟): AI 辅助架构设计

步骤	任务	时间
Step 1	让 LLM 生成在线书店的 PlantUML 包图	5 min
Step 2	让 LLM 生成 Spring Boot 目录结构	5 min
Step 3	审查 AI 输出 — 检查分层约束和依赖方向	5 min
Step 4	修正发现的问题, 记录改进点	5 min

审查 Checklist

- ☐ 依赖方向是否单向向下?
- ☐ 是否存在循环依赖?
- ☐ 接口和实现是否分离?
- ☐ entity 包是否只包含数据实体?
- ☐ 是否遵循 CCP / CRP 原则?

思政融入：可持续架构设计

技术债务的社会成本

技术债务产生原因

- 赶工期跳过架构设计
- 忽视包设计原则
- 不做集成测试
- 不做架构评审

社会层面影响

- 系统崩溃影响用户利益
- 维护成本转嫁给后续团队
- 安全漏洞危及数据安全
- 低质量软件浪费社会资源

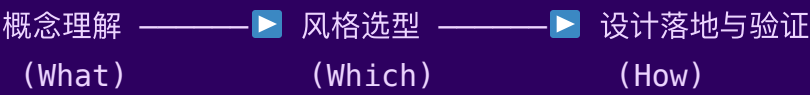
架构设计不仅是技术决策，更是一种**社会责任**。良好的架构设计是对未来维护者、用户和社会的负责。

PART 7 · 模块总结

体系结构模块三讲回顾

体系结构模块三讲回顾

讲次	主题	核心内容
03-01	软件设计基础与体系结构概念	设计本质、分解与抽象、体系结构三要素、风格分类
03-02	体系结构风格与设计决策	分层/MVC/管道-过滤器/微服务、质量属性、ADR
03-03	体系结构设计实践与验证	逻辑视角、4+1视图、接口设计、集成策略、AI辅助



核心能力总结

能力	知识点	实践要求
逻辑视角设计	4 步映射法、包设计六原则	能从用例导出包结构
多视图建模	4+1 视图模型	能画包图和部署图
接口设计	DIP、PO/VO、无状态设计	能设计 Service/Dao 接口
构建落地	目录映射、Spring Boot 配置	能搭建项目骨架
集成验证	Stub/Driver、CI、ATAM/SAAM	能选择集成策略并验证
AI 辅助	LLM Prompt 设计、输出审查	能利用 AI 加速架构设计

课后作业

作业要求

1. **包结构设计**：为"学生选课系统"设计完整的包结构（参考 4 步映射法），画出包图
2. **接口设计**：为选课系统编写 `CourseDao` 和 `CourseService` 接口（Java 代码）
3. **集成策略选择**：说明你会选择哪种集成策略，并给出理由
4. **AI 辅助**（附加题）：使用 LLM 生成选课系统的 Spring Boot 项目骨架，对比你的手工设计，记录差异和改进

提交要求

- 格式：PDF 文档
- 截止时间：下周上课前
- 提交方式：教学平台

体系结构设计实践与验证

软件工程与计算 II · 体系结构模块 · 第 3 讲

好的架构不是一次设计出来的，而是在实践中不断验证和演化的。

南京大学 · 刘钦